



# Synergetic attention-driven transformer: A deep reinforcement learning approach for vehicle routing problems

Qingshu Guan <sup>a,b</sup>, Hui Cao <sup>a,b</sup>, Lixin Jia <sup>a,b</sup>, Dapeng Yan <sup>a,b,\*</sup>, Badong Chen <sup>c,d</sup>

<sup>a</sup> School of Electrical Engineering, Xi'an Jiaotong University, Xi'an, 710049, China

<sup>b</sup> State Key Laboratory of Electrical Insulation and Power Equipment, Xi'an Jiaotong University, Xi'an, 710049, China

<sup>c</sup> National Key Laboratory of Human-Machine Hybrid Augmented Intelligence, Xi'an Jiaotong University, Xi'an, 710049, China

<sup>d</sup> Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University, Xi'an, 710049, China

## ARTICLE INFO

### Keywords:

Vehicle routing problem  
Deep reinforcement learning  
Multi-head attention  
Markov decision process

## ABSTRACT

Recently, deep reinforcement learning (DRL) has emerged as a promising avenue for tackling vehicle routing problems (VRPs). However, prevailing methods predominantly concentrate on learning construction policies, which often yield suboptimal outcomes in terms of generalization. In this study, we build upon an encoder-decoder framework and propose a synergetic attention-driven transformer (SAT) to acquire improvement policies, which strive to refine the current complete solution iteratively. During the encoding phase, we devise a multi-head synergetic attention mechanism to simultaneously learn node feature tokens and access index tokens in a dual pathway, thereby mitigating potential noises and incompatible correlations. Furthermore, we employ a cyclic mapping strategy to embed access sequences, enabling the capture of the inherent circularity and symmetry attributes within VRP solutions. In the decoding phase, we incorporate multiple decoders and formulate two regularization losses from perspectives of external policies and internal parameters to enhance the diversity of alternative solutions, thus broadening the search space efficiently. In the training phase, we utilize proximal policy optimization to ensure rapid and stable convergence of our model. Finally, we evaluate the efficacy of our method on two representative VRPs, namely the traveling salesman problem (TSP) and the capacitated VRP (CVRP). Comprehensive experiments on both synthetic and real-world benchmarks demonstrate that our SAT outperforms heuristic and DRL methods by a significant margin of up to 8.56% in precision, and also exhibits much better generalization performance on problems across varying sizes and distributions.

## 1. Introduction

Vehicle routing problems (VRPs) (Bogyrbayeva, Meraliyev, Mustakhov, & Dauletbayev, 2024) represent a fundamental class of combinatorial optimization problems (COPs) (Sun & Yang, 2024), which focus on determining the optimal route for efficiently traversing and delivering to a dispersed array of customer nodes under specific constraints. VRPs have garnered considerable attention for several decades owing to their profound theoretical implications in operation science and cybernetics and their ubiquitous practical applications, including transportation (Lei, Guo, Wang, Wu, & Zhao, 2022), robotics (Zhang, Zhao, Lamon, Wang, & Hong, 2023), electricity (Qiu, Wang, Ding, & Strbac, 2024), manufacturing (Zhang, Liu, Zhou & Wang, 2023) and so forth.

Traditional approaches for tackling VRPs can be broadly classified into two streams: exact methods and heuristic methods. Exact methods,

such as branch-cut-and-price (Lera-Romero, Bront, & Soullignac, 2024), offer compelling theoretical assurances of achieving optimal solutions, particularly for problems of moderate size. However, they grapple with formidable computational complexities when confronted with large-scale routing scenarios. In contrast, heuristic methods, exemplified by genetic algorithm (Li, Zhu, & Lee, 2023) and ant colony optimization (Qian, Liu, & Fang, 2024), leverage manually crafted rules to guide the search process and demonstrate proficiency in producing satisfactory solutions within reasonable computational timeframes. Nevertheless, the development of heuristics necessitates extensive trial-and-error iterations, and the efficacy of the solutions heavily depends on human expertise. Moreover, both exact and heuristic methods solve each instance in isolation, neglecting to discern crucial and potential patterns shared within the dataset.

\* Corresponding author.

E-mail addresses: [qsguan@stu.xjtu.edu.cn](mailto:qsguan@stu.xjtu.edu.cn) (Q. Guan), [huicao@mail.xjtu.edu.cn](mailto:huicao@mail.xjtu.edu.cn) (H. Cao), [lxjia@xjtu.edu.cn](mailto:lxjia@xjtu.edu.cn) (L. Jia), [dapengyan@xjtu.edu.cn](mailto:dapengyan@xjtu.edu.cn) (D. Yan), [chenbd@mail.xjtu.edu.cn](mailto:chenbd@mail.xjtu.edu.cn) (B. Chen).

<https://doi.org/10.1016/j.eswa.2025.126961>

Received 11 October 2024; Received in revised form 24 January 2025; Accepted 17 February 2025

Available online 27 February 2025

0957-4174/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

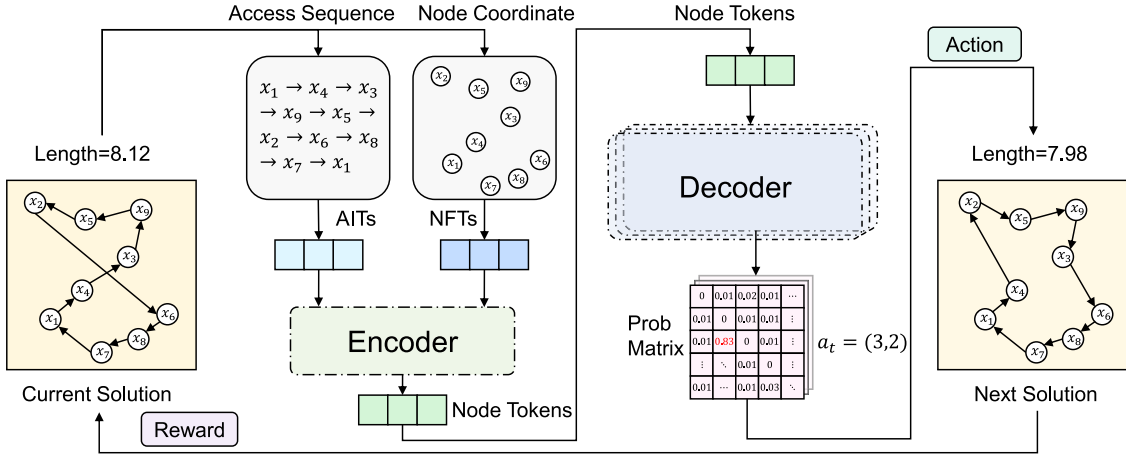


Fig. 1. An illustrative example of our proposed method. Given the node coordinates and access sequences defining the current solution, the encoder maps these raw features into high-dimensional enhanced tokens. Afterwards, the decoder leverages the compatibility block to generate a probability matrix, facilitating the selection of the most suitable node-pair for the application of 2-opt operators. As a result, the access sequence between  $x_2$  and  $x_3$  is reversed, leading to the attainment of an optimal solution for the routing problem.

Recently, in light of the rapid progress witnessed in deep neural networks spanning diverse domains (Jiao, Zhang, Zhang, & Li, 2024; Ma, Wang, et al., 2023; Wang, Nie, Zhang, Wang, & Li, 2023), there has been an escalating inclination towards employing deep reinforcement learning (DRL) (Lu, Li, & Wang, 2024; Sun, Wang, Lin, & Zhou, 2023; Zhu et al., 2023) to tackle various NP-hard VRPs, including the traveling salesman problem (TSP) (Zhang & Han, 2022) and capacitated VRP (CVRP) (Wang, Ma, & Sun, 2023). DRL conceptualizes the routing task as a Markov decision process (MDP) (Lauri, Hsu, & Pajarinen, 2022) and employs a sophisticated encoder-decoder framework to learn sequential decision-making strategies by discerning potential and pivotal patterns among problem instances. Remarkably, DRL methods obviate the reliance on prior domain-specific knowledge and exhibit the capability to achieve commendable routing performance on large-scale problems while maintaining satisfactory inference time. Prevailing DRL methods fall into two categories based on the solution generation process: construction-type DRL (Kool, van Hoof, & Welling, 2019; Kwon et al., 2020; Xu et al., 2021) and improvement-type DRL (Ma et al., 2022, 2021; Wu, Song, Cao, Zhang, & Lim, 2022). Among these, most of existing DRL methods learn a construction paradigm, wherein a complete solution is iteratively built by incorporating unvisited nodes into the current partial route at each step. Despite their relatively rapid execution speeds, their routing outcomes, particularly in terms of generalization, still exhibit a discernible gap relative to established COP solvers in objective values.

In this study, rather than focusing on constructive DRL, we attempt to explore a more intricate and efficacious paradigm, known as improvement-type DRL. This approach refines an initial complete solution by iteratively employing neural search via pair-wise node operators (e.g., 2-opt) to enhance solution quality. Improvement-type DRL methods learn policies to automatically optimize the node access sequence within the route, leading to superior performance in both inference and generalization, irrespective of problem size. Despite their achievements, prevailing improvement-type DRL methods still encounter several limitations. On the one hand, existing approaches commonly utilize conventional transformers, originally designed for natural language processing tasks, to directly encode node coordinates and access sequences. However, they overlook the intrinsic circularity attribute inherent in routing problems. The simplistic fusion of node coordinates and access sequences in self-attention mechanisms may introduce undesired noise and random biases to the network. On the other hand, the refined routes generated by the DRL model lack adequate

diversification. Recent insights from cognition and brain science (Luppi et al., 2022) indicate that humans who approach problems from multiple perspectives develop multi-modal problem-solving skills, thereby exhibiting remarkable performance on similar challenges. However, mainstream improvement-type approaches typically train a single policy discriminatively, neglecting the exploration of the entire solution space. This deficiency impedes further enhancements in routing performance.

To tackle these challenges, we build upon the encoder-decoder framework and propose a Synergetic Attention-driven Transformer (SAT), as illustrated in Fig. 1, to solve VRPs of varying sizes within an improvement paradigm. Firstly, we identify the circularity attribute intrinsic in routing solutions and design a novel cyclic mapping strategy to embed node access index tokens (AITs), thereby capturing the circularity and symmetry of routes and boosting the generalization performance effectively. Subsequently, we revisit the embedding representation and devise a multi-head synergetic attention (MHSA) mechanism to process both node feature tokens (NFTs) and AITs simultaneously in a dual pathway. This cooperative approach facilitates the discernment of consistence and differentiation between tokens, thus mitigating potential random biases. Drawing inspiration from cognitive research, we incorporate multiple decoders for node-pair operations and introduce two regularization losses from perspectives of external policy and internal parameters. These measures encourage decoders to generate diverse node-pair selection policies, consequently expanding the search space and enhancing routing outcomes efficiently. Finally, we conduct comprehensive experiments on both synthetic and real-world datasets to evaluate the proposed method. Experimental results demonstrate that our SAT significantly outperforms heuristic, construction-type DRL, and improvement-type DRL methods on TSPs and CVRPs. Furthermore, our SAT exhibits superior generalization performance across instances characterized by varying node distributions and problem sizes.

Our main contributions are summarized as follows:

- We build upon the encoder-decoder architecture and propose a SAT framework for learning improvement-type routing policies.
- In the encoding phase, we depict a cyclic mapping strategy to embed node access index tokens (AITs), thereby keenly capturing the circularity and symmetry attributes inherent in VRP routes.
- During decoding, we introduce multiple decoders and devise two regularization losses from perspectives of external policy and internal parameters to bolster the generation of a diverse array of alternative solutions.

- Extensive experimental results demonstrate that our proposed method outperforms a variety of heuristic and DRL methods on both synthetic and real-world instances across different sizes and distributions.

## 2. Related work

### 2.1. Traditional methods for VRPs

VRPs have been a focal point of extensive research over decades, given their profound implications in both theoretical and practical domains within computer science and operations research (Jiang, Dai, Yang, & Ma, 2024; Ren et al., 2022; Zhou et al., 2023). A multitude of inventive approaches has been proposed to address the inherent NP-hard nature of these challenges. Within this spectrum, traditional routing methodologies fall into two categories: exact and heuristic solvers. Exact solvers, exemplified by branch-and-cut (Tamke & Buscher, 2021) and its variants (Jiang & Zhang, 2022), formulate VRPs as mixed-integer programming problems and leverage bound estimation to expedite the exhaustive search process. Additionally, the mature solver Concorde (Applegate, Bixby, Chvátal, & Cook, 2020) is also guaranteed to achieve the optimal solution tailored specifically for TSPs. Despite their prowess, exact approaches grapple with the inherent challenge of high computational complexity. Specifically, as the scale of routing problems increases, the execution time of exact methods experiences an exponential surge, curtailing their applicability in expansive, large-scale scenarios.

In contrast, heuristic methods excel in narrowing the search space through meticulously crafted rules, striking a favorable trade-off between solution quality and execution time. Representative heuristic algorithms, encompassing genetic algorithm (GA) (Hussain et al., 2017), simulated annealing (SA) (Zhou, Xu, Fu, & Zhou, 2022), ant colony optimization (ACO) (Hlaing & Khine, 2011), tabu search (TS) (Gmira, Gendreau, Lodi, & Potvin, 2021), and Lin-Kernighan-Helsgaun (LKH) heuristics (Zheng, He, Zhou, Jin, & Li, 2023), utilize manually designed features to guide the exploration process, consistently yielding superior routing performance. Evolutionary computation-based GA (Hussain et al., 2017) employs a chromosomal coding structure to establish the initial population of solutions, progressively enhancing its quality through biologically inspired operators such as mutation, crossover, and selection. Swarm intelligence-based ACO (Hlaing & Khine, 2011) exploits pheromone communication among artificial ants to comprehensively explore the entire solution space, demonstrating remarkable prowess in addressing complex optimization problems like vehicle routing and network reconfiguration. Moreover, versatile mathematical optimization solvers, like Gurobi (Optimization & LLC, 2022) and Google OR Tools (Perron & Furnon, 2019), integrate a spectrum of heuristic algorithms and showcase adaptability and robustness in resolving various VRPs, particularly in complex real-world scenes. Nevertheless, it is essential to note that the efficacy of such heuristic methods requires substantial trial-and-error, and the ultimate solution quality heavily relies on expert intuition and domain-specific knowledge.

### 2.2. DRL-based methods for VRPs

Recently, there has been a discernible surge in the application of neural methodologies to address VRPs, particularly within the ambit of deep (reinforcement) learning architectures (Jiang, Cao, Wu, Song, & Zhang, 2023; Tang et al., 2023; Ye et al., 2024). Unlike traditional operations research approaches that model VRPs as mixed-integer programming problems, neural DRL-based methods reframe them as MDPs and leverage neural networks to uncover underlying patterns and guide routing decisions. Depending on their approach to solution generation, these neural methods can be broadly divided into two categories: construction methods and improvement methods.

#### 2.2.1. Neural construction methods for VRPs

Neural construction methods, commencing with an empty solution, progressively incorporate unvisited nodes into the evolving tour to construct the complete route. Notably, Bello, Pham, Le, Norouzi, and Bengio (2017) employ reinforcement learning to train the Pointer Network (Ptr-Net) (Vinyals, Fortunato, & Jaitly, 2015), yielding commendable results in addressing TSPs. Simultaneously, Kool et al. (2019) harness the encoder-decoder framework and propose a milestone transformer-based Attention Model (AM) proficient in resolving diverse routing problems. Furthermore, AM incorporates a multi-head attention mechanism and a greedy rollout estimator to enhance the effectiveness and efficiency of model training. Building upon this foundation, Kwon et al. (2020) recognize the symmetry inherent in routing challenges and introduce instance augmentation to bolster policy optimization with multiple optima (POMO). It is noteworthy that POMO achieves state-of-the-art routing performance with rapid solving speeds. Subsequently, a myriad of neural construction methods (Bogyrbayeva et al., 2023; Fan et al., 2022; Li et al., 2021) has been proposed to tackle a variety of VRPs. Despite their comparative speed, their results still exhibit a discernible margin when juxtaposed against well-established exact approaches in terms of objective values.

#### 2.2.2. Neural improvement methods for VRPs

In a distinct avenue of research, neural improvement methods, commencing from a fully constructed initial solution, aim to refine the existing tour through the strategic application of DRL for guiding and assisting the neighbor search processes. Specifically, improvement-type models (Kim, Park, et al., 2021; Lu, Zhang, & Yang, 2019; Ma, Cao, & Chee, 2023) utilize certain local operators, such as 2-opt, node swap, and relocate, to incrementally enhance the solution quality. Pioneering this approach, Chen and Tian (2019) employ region-picking and rule-picking policies in their model, NeuRewriter, facilitating the local rewriting of components towards route optimization. NeuRewriter, characterized by its profound understanding of combinatorial optimization problems, demonstrates robust performance in robot routing, job scheduling, and expression simplification. Furthermore, Wu et al. (2022) introduce a transformer-based routing model designed to learn pair-wise operators, surpassing NeuRewriter in performance on TSPs and CVRPs. Additionally, Ma et al. (2021) conceptualize a dual-aspect collaborative transformer (DACT) for independently learning embeddings of nodal and positional features. Remarkably, DACT exhibits significantly superior performance compared to other transformer-based improvement methods. In a more recent work, DACT undergoes further modification to learn removal and reinsertion operators tailored for pickup-delivery problems (Ma et al., 2022). Nevertheless, prevailing improvement-type methods are still hindered by the challenge of restricted exploration within the search space, which ultimately inhibits the potential for enhanced solution quality.

## 3. Methodology

### 3.1. Formulation

In this paper, we delineate the problem in the context of TSP, while noting that the input and formulation can be subtly adapted for other VRPs following (Kool et al., 2019; Kwon et al., 2020; Li et al., 2021). The input for the proposed model is characterized by an order-invariant node coordinate matrix  $X = [x_1; x_2; \dots; x_N] \in \mathbb{R}^{N \times d_x}$ , wherein each node encompasses a  $d_x$ -dimensional coordinate and  $N$  signifies the total number of nodes. The desired output entails a sequence of nodes  $\Psi = (p_1, p_2, \dots, p_N)$ , where the access index  $p_i$  designates that the node  $x_i$  is the  $p_i$ -th one to be visited in the solution. The overarching objective is to minimize the total length of the traversed route while adhering to specific constraints inherent to the given problem.

Starting from an initial yet complete solution  $\Psi_0$ , our DRL model, grounded in the improvement paradigm, systematically endeavors to

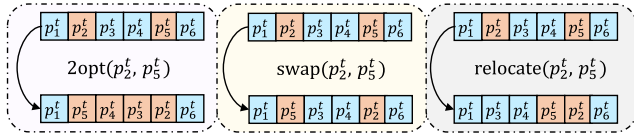


Fig. 2. Illustration of three pair-wise node operators for routing problems (left: 2-opt reverses a segment of access sequence; middle: node swap exchanges two visiting locations; right: relocation adjusts one location after another location).

refine the solution, focusing on the sequence of node accesses. At each time step, the model autonomously selects a pair of nodes from the overall graph, and locally adjusts the solution through the utilization of a predefined pair-wise node operator like 2-opt, node swap, and relocate, as illustrated in Fig. 2. Our model chooses 2-opt as operator in this paper since it achieves best comparison results, as demonstrated in Table 5 in Section 4.4.1. The node-centric operation iterates until termination, and we conceptualize this incremental improvement process as a MDP. This formulation aligns seamlessly with the principles of DRL and proves to be highly effective in its application.

### 3.1.1. State

The state  $s_t$  denotes the current solution to an given instance at time step  $t$ , which consists of time-invariant node coordinates  $\{x_i\}_{i=1}^N$  and a time-varying node access sequence  $\Psi_t = \{p_i^t\}_{i=1}^N$ . The solution  $\Psi_t$  represents a sequential visitation of nodes, where  $p_i^t$  serves as the access index of node  $x_i$  within the tour. Consequently, the state  $s_t$  can be articulated as a combination of both node and index features, formally expressed as  $s_t = \left\{ \{x_i\}_{i=1}^N, \{p_i^t\}_{i=1}^N \right\}$ .

### 3.1.2. Action

As our objective is to portray a solution within the neighborhood delineated by local pair-wise operators, the action  $a_t$  precisely designates a chosen node pair  $(i, j)$  for executing operations such as 2-opt, swap, and relocate.

### 3.1.3. Transition

The next state  $s_{t+1}$  arises from the current state  $s_t$  through the execution of the predetermined operator on the specified node pair  $a_t = (i, j)$ . The probability of state transition is articulated as  $p(s_{t+1}|a_t, s_t) = 1$ . This deterministic transition rule facilitates the model in acquiring the ability to synergize multiple local pair-wise movements effectively, leading to the achievement of satisfactory results.

### 3.1.4. Reward

In improvement-type models, the primary objective is to optimize the initial solution to the greatest extent within a finite number of steps. Accordingly, the reward  $r_t$  is defined as the immediate reduced cost at each time step relative to the prevailing optimal solution, and the cumulative reward precisely quantifies the decrease in cost compared to the initial solution. Specifically,  $r_t = C(\Psi_t^*) - \min(C(\Psi_{t+1}) - C(\Psi_t^*))$ , where  $\Psi_t^*$  indicates the best incumbent solution identified up to time step  $t$  and  $C(\cdot)$  signifies the cost (i.e. tour length) of the solution.

### 3.1.5. Policy

Commencing from the initial state  $s_0$ , the stochastic policy  $p_\theta$ , parameterized by  $\theta$ , chooses an action  $a_t$  at each time step  $t$ , thereby navigating to the subsequent state  $s_{t+1}$  until reaching the time step limit  $T$ . This iterative progression can be elucidated through the application of a probability chain rule:

$$p_\theta(s_T|s_0) = \prod_{t=0}^{T-1} p_\theta(a_t|s_t)p(s_{t+1}|a_t, s_t), \quad (1)$$

where  $p(s_{t+1}|a_t, s_t) = 1$  since we employ a deterministic state transition rule. The policy endows our model with the capacity to make judicious pair-wise decisions at each time step, culminating in an optimal solution in the improvement-type routing process.

## 3.2. Overall framework

Our proposed SAT utilizes an improvement-type DRL to refine the current solution iteratively. Aligned with the prevailing transformer-based framework, SAT incorporates an encoder for node embedding, a decoder for node pair selection, and a training algorithm for model optimization. To effectively capture the inherent characteristics of the current state and achieve superior route refinement, our model undergoes two key modifications. (1) We discern the circularity and symmetry inherent in routing solutions, and integrate node access indices alongside node features to construct high-quality node tokens using a dual-pathway synergetic attention mechanism. (2) We introduce multiply lightweight decoders and design two regularization losses (focusing on external policy and internal parameter aspects) to encourage the generation of diverse routes, thereby facilitating broader exploration of the solution space and leading to enhanced routing performance. Accordingly, we employ an actor-critic algorithm with proximal policy optimization to train the routing model, ensuring smooth gradient updates and rapid network convergence.

### 3.3. Encoder

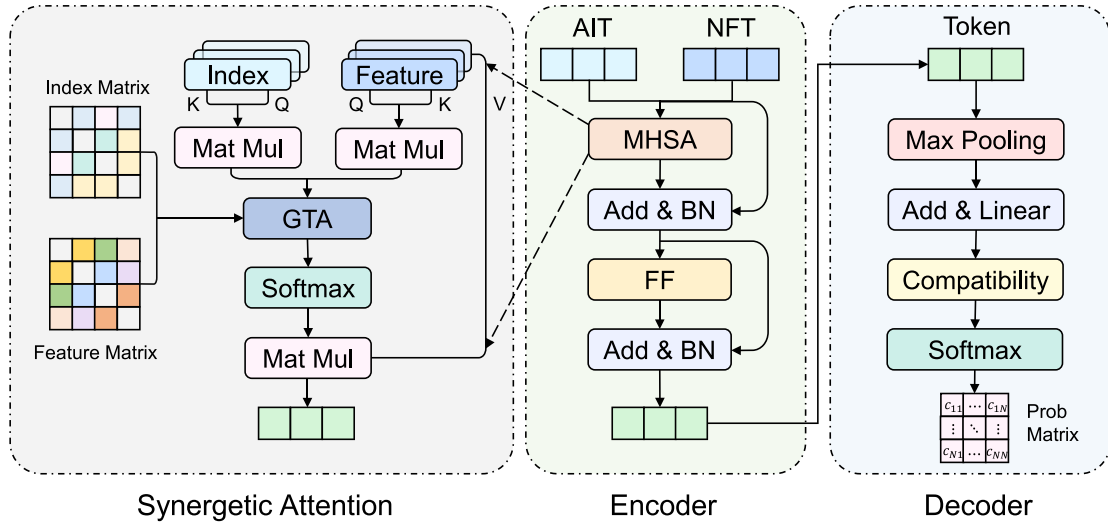
Given the current state  $s_t$ , the proposed encoder utilizes node features  $\{x_i\}_{i=1}^N$  and access indices  $\{p_i^t\}_{i=1}^N$  as inputs to acquire tokens for encapsulating the present solution. Notably, the time-varying access indices of nodes serve as specialized features tailored for improvement-type routing. Initially, we project these raw features through a dual pathway, yielding two distinct sets of high-dimensional tokens: node feature tokens (NFTs) and access index tokens (AITs). On the one hand, we transform node features  $X$  into representative NFTs  $H^{(0)}$  through a linear projection:  $H^{(0)} = XW^x + B^x$ , where  $W^x \in \mathbb{R}^{d_x \times d_h}$  and  $B^x \in \mathbb{R}^{N \times d_h}$  are parameter matrices. On the other hand, we discern the circularity and symmetry attributes inherent in routing solutions, and introduce a cyclic mapping approach to construct AITs  $G = [g_1; g_2; \dots; g_N] \in \mathbb{R}^{N \times d_g}$ . This innovative approach draws inspiration from positional embedding techniques prevalent in natural language processing (NLP) (Su et al., 2024) and robotics (Guan et al., 2021).

$$g_i^{(d)} = \begin{cases} \frac{1}{2} + \frac{1}{2} \sin\left(\frac{2\pi}{T_d} \cdot |p_i - T_d|\right), & \text{if } d \text{ is even} \\ \frac{1}{2} + \frac{1}{2} \cos\left(\frac{2\pi}{T_d} \cdot |p_i - T_d|\right), & \text{if } d \text{ is odd,} \end{cases} \quad (2)$$

$$T_d = \begin{cases} \frac{3\lfloor d_g/3+1 \rfloor}{d_g} \left( N - N^{\lfloor d_g/2 \rfloor} \right) + N^{\frac{1}{\lfloor d_g/2 \rfloor}}, & \text{if } d < \lfloor \frac{d_g}{2} \rfloor \\ N, & \text{otherwise,} \end{cases} \quad (3)$$

where  $g_i^{(d)} \in [0, 1]$  represents the value in the  $d$ th dimension of vector  $g_i$ , and the angular frequency  $\frac{2\pi}{T_d}$  decreases progressively across dimensions, aiming to align the wavelength within the range  $\left[ N^{\frac{1}{\lfloor d_g/2 \rfloor}}, N \right]$ , as elucidated in existing literature (Ma et al., 2021).

Considering that directly fusing the two types of tokens may induce undesired noises in conventional self-attention mechanisms, we prioritize NFTs as dominant tokens for primary feature representation, while AITs serve as supplementary indicators for the visiting sequence. Building upon this foundation, we design a streamlined and efficient Multi-Head Synergetic Attention (MHSA) mechanism, facilitated by a meticulously crafted Gated Token Aggregation (GTA) block. As shown in Fig. 3, this approach not only harnesses primary self-attention correlations derived from NFTs, but also integrates auxiliary self-attention correlations learned from AITs. These correlations are then channeled into the GTA block, where they are synthesized into a comprehensive attention mechanism for both contradistinction and collaboration. Overall, the innovative MHSA not only learns polyphyletic attention correlations from various tokens, but achieve superior performance with reduced computational overhead as well. The calculation of the



**Fig. 3.** Architecture of the policy network. Our approach adopts an encoder–decoder framework as the backbone. In this setup, the encoder learns synergetic attention from AITs and NFTs through a dual pathway, facilitating the embedding of raw features into high-dimensional tokens. This process enhances feature extraction and characterization capabilities, ensuring robustness in representation learning. On the other hand, the lightweight decoder utilizes the compatibility block to discern potential associations between nodes, thereby generating a probability matrix dictating the selection of each node pair. Consequently, the route undergoes iterative modifications via node-pair operators, progressively converging towards optimal solutions.

MHSA block proceeds as follows:

$$\tilde{H} = \text{MHSA}(H, G). \quad (4)$$

We initially compute the attention correlations  $Z_m^h$  and  $Z_m^g$  for NFTs and AITs by facilitating information transfer and interaction:

$$Z_m^h = \frac{1}{\sqrt{d_k}} (H W_m^{Q_h}) (H W_m^{K_h})^\top, \quad (5)$$

$$Z_m^g = \frac{1}{\sqrt{d_k}} (G W_m^{Q_g}) (G W_m^{K_g})^\top, \quad (6)$$

where  $W_m^{Q_h}, W_m^{K_h} \in \mathbb{R}^{d_h \times d_k}$  and  $W_m^{Q_g}, W_m^{K_g} \in \mathbb{R}^{d_g \times d_k}$  are *query* and *key* matrices with learnable parameters for each head  $m$ . It is essential to highlight that these attention correlations are computed independently, thereby mitigating potential noise and fostering autonomy among tokens from various perspectives. Next, we leverage the GTA block to amalgamate dual-pathway attention correlations into a unified form  $Z_m$ , aiming to enhance feature representation.

$$Z_m = \text{GTA}(Z_m^h, Z_m^g) \\ = Z_m^h + \sigma(Z_m^h W^G + B^G) \odot Z_m^g, \quad (7)$$

where  $\sigma(\cdot)$  denotes the sigmoid function,  $\odot$  signifies the Hadamard element-wise product, and  $W^G, B^G \in \mathbb{R}^{N \times N}$  are trainable parameter matrices. The synthesized relationships are then standardized through a row-wise Softmax operation to derive the attention values. The resultant output  $\tilde{H}$  of MHSA block is concatenated with  $M$  heads and projected through the matrix  $W^O \in \mathbb{R}^{M d_v \times d_h}$ :

$$E_m = \text{Softmax}(Z_m) H W_m^{V_h}, \quad (8)$$

$$\tilde{H} = \text{Concat}[E_1, \dots, E_M] W^O, \quad (9)$$

where  $W_m^{V_h} \in \mathbb{R}^{d_h \times d_v}$  is a learnable *value* matrix with  $d_v = d_h/M$ .

Accordingly, taking a holistic view, our encoder comprises  $L$  stacked attention layers. Each layer incorporates a MHSA sub-layer and a feed-forward (FF) sub-layer, supplemented with skip connection and batch normalization.

$$\hat{H}^{(l)} = \text{BN}(H^{(l-1)} + \text{MHSA}(H^{(l-1)}, G)), \quad (10)$$

$$H^{(l)} = \text{BN}(\hat{H}^{(l)} + \text{FF}(\hat{H}^{(l)})), \quad (11)$$

where  $\text{BN}(\cdot)$  denotes the batch normalization operator,  $\text{FF}(\cdot)$  represents the FF operator, which includes two linear fully connected layers separated by a rectified linear unit (ReLU). The synthesized token  $H^{(l)}$  denotes the output of the  $l$ th layer and serves as the input of the  $(l+1)$ -th layer.

### 3.4. Decoder

The proposed decoder initially incorporates the max-pooling operation as described in Wu et al. (2022), aiming to aggregate the global representation features of all node tokens into each local token individually. This process generates enhanced contextual tokens as follows:

$$h_i^c = h_i^{(L)} W^{\text{Local}} + \max \left\{ \left\{ h_i^{(L)} \right\}_{i=1}^N \right\} W^{\text{Global}}, \quad (12)$$

where  $W^{\text{Local}}, W^{\text{Global}} \in \mathbb{R}^{d_h \times d_h}$  are trainable projection matrices and  $h_i^{(L)}$  denotes the  $i$ th row of the token  $H^{(L)}$ . Through this approach, the global graph information is intricately embedded into each node token.

Given the enhanced tokens for each node, the decoder proceeds to refine them through a compatibility layer and a masked softmax layer to ascertain the likelihood of selecting each node pair. Drawing inspiration from the application of compatibility (Vaswani et al., 2017) in NLP, where it characterizes the relationships between words in sentences, we utilize the multiplicative version to anticipate the selection of node pairs with 2-opt within a complete solution. Our compatibility layer offers two advantages: (1) its computational speed is accelerated due to highly optimized matrix multiplication with PyTorch, and (2) it consumes fewer GPU resources as it does not require additional deep neural networks. Specifically, the compatibility  $u_{ij}$  is calculated as:

$$u_{ij} = \begin{cases} \mathcal{E} \tanh \left( \frac{1}{\sqrt{d_k}} h_i^c W^{Q_c} (h_j^c W^{K_c})^\top \right), & \text{if } i \neq j \\ -\infty, & \text{otherwise} \end{cases}, \quad (13)$$

where the constant  $\mathcal{E}$  is employed to regulate the entropy of  $u_{ij}$  within  $[-\mathcal{E}, \mathcal{E}]$ ,  $\tanh(\cdot)$  indicates the hyperbolic tangent function, and  $W^{Q_c}, W^{K_c} \in \mathbb{R}^{d_h \times d_k}$  represent learnable *query* and *key* matrices. Subsequently, the compatibility  $u_{ij}$  is passed through the softmax layer to generate the probability of selecting the node pair  $(i, j)$  for local 2-opt

operation:

$$c_{ij} = p_\theta(a_i = (i, j) | s_t) = \frac{e^{u_{ij}}}{\sum_{i,j} e^{u_{ij}}}. \quad (14)$$

However, employing a single decoder restricts training to a singular strategy, resulting in the generation of partially similar solutions. This setting constrains the exploration of the search space and increases the likelihood of the model becoming trapped in local optima. To address this issue, we introduce multiple decoders with identical structures but unshared parameters to output a more diversified array of routes and thoroughly explore the entire solution space. Specifically, we attempt to foster diversity from both *external* policy and *internal* parameter perspectives. On one hand, we design a regularization term  $\mathcal{L}_{\text{EXT}}$  based on Kullback–Leibler divergence to encourage multiply decoders to adopt different external policies (i.e. node-pair selections) and strike a better balance between exploration and exploitation.

$$\mathcal{L}_{\text{EXT}} = -\frac{1}{N^2 Z^2} \sum_{z=1}^Z \sum_{z'=1}^Z \sum_{i=1}^N \sum_{j=1}^N c_{ij}^z \log \frac{c_{ij}^z}{c_{ij}^{z'}}, \quad (15)$$

where  $c_{ij}^z$  is the probability for decoder  $z$  to select the node pair  $(i, j)$  and  $Z$  denotes the total number of decoders.

On the other hand, we further incentive the exploration of diversification concerning the internal network parameters among different decoders during training. We employ the Theil index (Tian, Zhao, & Yuan, 2021), a statistical measure widely used in economics to estimate inequality within a population, to assess the parameter entropy of multiply decoders. Through the application of the Theil index, we can ascertain whether our model acquires distinct knowledge or merely mimics each other. Concretely, we depict the loss function  $\mathcal{L}_{\text{INT}}$  based on the Theil index as:

$$\mathcal{L}_{\text{INT}} = -\frac{1}{Z} \sum_{z=1}^Z \frac{\delta_z}{\mu} \ln \frac{\delta_z}{\mu}, \quad (16)$$

where  $\delta_z$  denotes the  $\ell_2$ -norm of parameter  $\theta_z$  and  $\mu$  represents the mean of all  $\ell_2$ -norms.

### 3.5. Training

Given the current state, our proposed model iteratively improves the traversal tour by employing node pair operators. We leverage the proximal policy optimization (PPO) with  $n$ -step return estimation for model training. In addition, we incorporate a curriculum learning (CL) strategy, following the work (Ma et al., 2021), to further ameliorate the convergence process.

More specifically, we employ the actor–critic algorithm, which integrates an actor network  $p_\theta$  and a critic network  $v_\phi$ , to mitigate variance during policy gradient updates. The actor network encompasses a reinforcement term  $J_{\text{RL}}$  for solution optimization and two regularization losses  $\mathcal{L}_{\text{EXT}}, \mathcal{L}_{\text{INT}}$  for route diversity. Furthermore, the critic network takes NFTs and AITs as input and enhances them through a vanilla MHA block (with 4 heads). The enhanced tokens are passed through a pooling layer (similar to Eq. (12)), and subsequently processed by a 4-layer FF network (with 129, 128, 64, and 1 units) to obtain the output value  $v_\phi(\cdot)$ .

$$J_{\text{Actor}}(\theta) = J_{\text{RL}}(\theta) - \mathcal{L}_{\text{EXT}} - \mathcal{L}_{\text{INT}}, \quad (17)$$

$$J_{\text{RL}}(\theta) = \frac{1}{n |D_{\text{batch}}|} \sum_{D_{\text{batch}}} \sum_{\tau=t}^{t+n} \min \left( \frac{p_\theta(a_\tau | s_\tau)}{p_{\theta_{\text{old}}}(a_\tau | s_\tau)} \hat{A}_\tau, \right. \\ \left. \text{clip} \left[ \frac{p_\theta(a_\tau | s_\tau)}{p_{\theta_{\text{old}}}(a_\tau | s_\tau)}, 1 - \epsilon, 1 + \epsilon \right] \hat{A}_\tau \right), \quad (18)$$

$$J_{\text{Critic}}(\theta) = \frac{1}{n |D_{\text{batch}}|} \sum_{D_{\text{batch}}} \sum_{\tau=t}^{t+n} \max \left( |v_\phi(s_\tau) - \hat{R}_\tau|, \right. \\ \left. |v_\phi^{\text{clip}}(s_\tau) - \hat{R}_\tau|^2 \right), \quad (19)$$

### Algorithm 1: Training Process of $n$ -step PPO with the CL Strategy

---

**Input** : policy network  $p_\theta$ , critic network  $v_\phi$ , number of epochs  $E$ , number of batches  $B$ , number of mini-batches  $K$ , number of training steps  $T_{\text{train}}$ , CL scalar  $\kappa$ , reward discount factor  $\gamma$ , learning rate  $\eta_\theta, \eta_\phi$ , decay rate  $\zeta$ , clipping threshold  $\epsilon$ .

---

```

1 Construct the training set  $D_{\text{train}}$ ;
2 for  $e = 1, 2, \dots, E$  do
3   for  $b = 1, 2, \dots, B$  do
4     Sample a batch of training instances  $D_b$  randomly;
5     CL Strategy: Generate initial solutions  $\{\Psi_b\}$  to dataset  $D_b$ ;
6     Improve  $\{\Psi_b\}$  to  $\{\Psi'_b\}$  via policy  $p_\theta$  for  $T = e/\kappa$  steps;
7     Construct the initial state  $s_0$  based on solutions  $\{\Psi'_b\}$ ;
8     Set  $t \leftarrow 0$ ;
9     while  $t < T_{\text{train}}$  do
10      PPO: Gather experience  $\{(s_\tau, a_\tau, r_\tau)\}_{\tau=t}^{t+n}$  by running
11      the actor network  $p_\theta$  for  $n$  time steps according to
12      Eq. (1);
13      Set  $t \leftarrow t + n$ ,  $p_{\theta_{\text{old}}} \leftarrow p_\theta$ ,  $v_{\phi_{\text{old}}} \leftarrow v_\phi$ ;
14      for  $k = 1, 2, \dots, K$  do
15         $\hat{R}_{t+1} = v_\phi(s_{t+1})$ ;
16        for  $\tau \in \{t, t-1, \dots, t-n\}$  do
17           $\hat{R}_\tau \leftarrow r_\tau + \gamma \hat{R}_{\tau+1}$ ;
18           $\hat{A}_\tau \leftarrow \hat{R}_\tau - v_\phi(s_\tau)$ ;
19        end
20        Calculate the PPO-driven actor network loss
21         $J_{\text{Actor}}(\theta)$  via Eqs. (15)–(18) and the clipped
22        critic network loss  $J_{\text{Critic}}(\theta)$  via Eqs. (19)–(20);
23         $\theta \leftarrow \theta + \eta_\theta \nabla J_{\text{Actor}}(\theta)$ ;
24         $\phi \leftarrow \phi - \eta_\phi \nabla J_{\text{Critic}}(\theta)$ ;
25      end
26    end
27  end
28   $\eta_\theta \leftarrow \zeta \eta_\theta$ ,  $\eta_\phi \leftarrow \zeta \eta_\phi$ ;
29 end
```

---

$$v_\phi^{\text{clip}}(s_\tau) = \text{clip} \left[ v_\phi(s_\tau), v_{\phi_{\text{old}}}(s_\tau) - \epsilon, v_{\phi_{\text{old}}}(s_\tau) + \epsilon \right]. \quad (20)$$

As delineated in Algorithm 1, our model undergoes training for  $E$  epochs with  $B$  batches per epoch, wherein the training set  $D_{\text{batch}}$  is generated on the fly. To expedite training, we introduce the CL strategy for state initialization. This strategy enhances solutions  $\{\Psi_b\}$  to  $\{\Psi'_b\}$  over  $T$  steps, and the resultant high-quality solutions are then utilized to initialize the first state  $s_0$ . Consequently, the difficulty level of neural search increases with each epoch, thereby facilitating smoother and more rapid iterations. Furthermore, we exploit the  $n$ -step PPO to ensure stable updates of the actor network  $p_\theta$  and critic network  $v_\phi$  in accordance with Eqs. (15)–(20).

## 4. Experiments

In this section, we assess the efficacy of our proposed model in comparison with a variety of heuristic and DRL-based methods encompassing construction and improvement strategies. Our evaluation encompasses experiments conducted on synthetic instances featuring diverse node distributions, as well as real-world instances sourced from two prominent datasets TSPLIB and CVRPLIB. All methodologies are meticulously implemented in PyTorch, leveraging the computational capabilities of an Intel Core i9-10900K @ 3.70 GHz CPU and a single Nvidia GeForce RTX 2080Ti GPU.

#### 4.1. Experimental setups

##### 4.1.1. Configuration

In this study, we undertake experiments on two prototypical routing conundrums, namely TSP and CVRP, across both uniform-distribution and cross-distribution scenarios to ascertain the effectiveness of our SAT. For uniform-distribution scenarios, we adhere to prevailing conventions (Kool et al., 2019; Kwon et al., 2020; Ma et al., 2021; Wu et al., 2022) to generate instances comprising 20, 50, and 100 nodes for TSP and CVRP, which are denoted as TSP20, CVRP20, and so forth for clarity. Node coordinates within instances are randomly sampled from a two-dimensional uniform distribution spanning the interval  $[0, 1]$  for both dimensions. Moreover, we stipulate vehicle capacities of 30, 40, and 50 for CVRP20, CVRP50, and CVRP100, respectively, with the demand of each node uniformly sampled from integers  $\{1, 2, \dots, 9\}$ .

To explore cross-distribution scenarios, we broaden our investigation to encompass five additional distribution types, including *expansion*, *implosion*, *explosion*, *grid*, and *cluster*, as shown in Fig. 4, and follow the literature (Jiang, Wu, Cao, & Zhang, 2022) to generate instances with 50 and 100 nodes for each problem. Specifically, we construct typical uniform-distribution instances as the predominant group and select one of the five distributions to generate atypical instances as the minority group. Consequently, five combinations of cross-distribution instances are formed and utilized for both training and testing across TSPs and CVRPs. Furthermore, we maintain consistent settings for vehicle capacity and node demand for CVRPs across varying distribution scenarios.

##### 4.1.2. Hyperparameters

In order to ensure a fair comparison with prevailing improvement-type DRL methods (Ma et al., 2021; Wu et al., 2022), we maintain consistency in network hyperparameters. Raw features are mapped into 128-dimensional tokens through a linear projection. The encoder of SAT comprises 3 stacked layers, each incorporating 8 attention heads for the MHSA sub-layer and a hidden dimension of 512 for the FF sub-layer. We opt for a configuration of five decoders. Compatibility values are constrained within the interval  $[-6, 6]$  as stipulated in Eq. (13). Model training spans 200 epochs, with 20 batches per epoch and a batch size of 600 for TSPs and CVRPs. We employ  $n = 5$  and  $T_{\text{train}} = 250$  for the  $n$ -step PPO, and utilize  $K = 3$  mini-batch updates along with a clip threshold  $\epsilon = 0.1$  as defined in Eq. (18) and (20). We set the reward discount factor  $\gamma = 0.999$ , and adopt the Adam optimizer to optimize the model with learning rates  $\eta_\theta = 1 \times 10^{-4}$  and  $\eta_\phi = 3 \times 10^{-5}$ , both of which decay by  $\zeta = 0.985$  per epoch to facilitate convergence. Additionally, the CL scalar  $\kappa$  is set to 2, 1.5, and 1 for three problem sizes, respectively.

##### 4.1.3. Training

We perform routing experiments across both uniform-distribution and cross-distribution scenarios. For investigating our model's efficacy in uniform-distribution scenarios, we train it on 100,000 synthetic instances characterized by uniform nodal distribution. Subsequently, we evaluate its efficacy on an additional set of 10,000 independent instances, preserving the same uniform distribution. For the more challenging cross-distribution scenarios, the training set encompasses five pairs of distribution groups. Each pair consists of 100,000 typical instances characterized by uniform distribution and 10,000 atypical instances following one of the other five distributions, including expansion, implosion, explosion, grid, and cluster. The test set comprises 10,000 independent instances generated with uniform distribution, along with 1,000 atypical instances adhering to the same distribution used during training. To ensure equitable comparisons, all learning-based baseline methods undergo training and testing on the same instance sets as our model. Furthermore, heuristic methods, devoid of training requirements, are directly applicable for testing purposes.

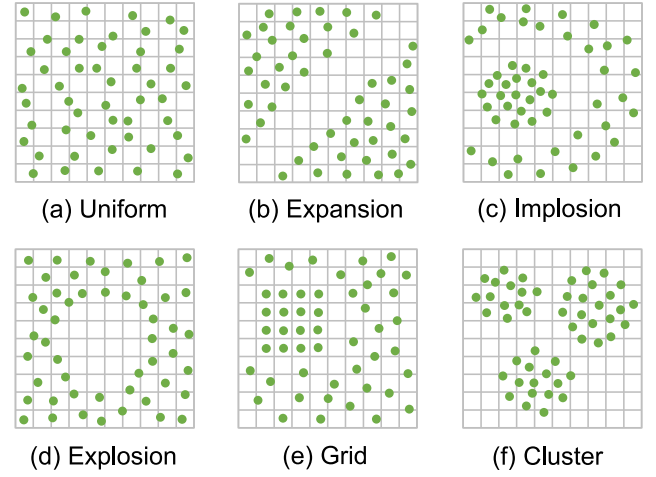


Fig. 4. An example illustration from our experimental setups with VRP instances characterized by six distinct distributions drawn from the literature (Jiang et al., 2022): (a) Uniform; (b) Expansion; (c) Implosion; (d) Explosion; (e) Grid; (f) Cluster.

##### 4.1.4. Evaluation metrics

We assess the performance of exact, heuristic, and DRL-based methods according to evaluation metrics, including the objective value (average path length), optimality gap, and average running time. Specifically, the optimality gap is quantified as the ratio difference between the average tour length  $L_{\text{aver}}$  across the test set and the optimal tour length  $L_{\text{opt}}$  obtained through the exact Concorde (Applegate et al., 2020) for TSPs and state-of-the-art LKH (Helsgaun, 2017) for CVRPs.

$$\text{Gap} = \frac{L_{\text{aver}} - L_{\text{opt}}}{L_{\text{opt}}} \times 100\%. \quad (21)$$

#### 4.2. Comparison analysis

We conduct a comprehensive comparison of our SAT with a spectrum of baseline methods, including: (1) Concorde (Applegate et al., 2020), a well-established exact solver tailored for TSPs; (2) LKH (Helsgaun, 2017), a highly efficient heuristic solver renowned for its cutting-edge performance on various routing problems; (3) Gurobi (Optimization & LLC, 2022), a commercial mathematical optimizer widely utilized for addressing combinatorial optimization problems; (4) OR Tools (Perron & Furnon, 2019), open-source and versatile operations research tools developed by Google; (5) random/nearest/farthest insertion and nearest neighbor: non-learned node selection methods recognized for their cost-effectiveness; (6) heuristic paradigms, such as GA (Hussain et al., 2017), SA (Hottung & Tierney, 2020), and ACO (Hlaing & Khine, 2011), which rely on human-designed rules to guide the route search process; (7) construction-type DRL methods, i.e., AM (Kool et al., 2019) and POMO (Kwon et al., 2020), which harness attention mechanisms to tackle VRPs with competitive performance; (8) improvement-type DRL methods, encompassing Wu et al. (2022), DACT (Ma et al., 2021), Neural-2-Opt (d O Costa, Rhuggenaath, Zhang, & Akcay, 2020), NLNS (Hottung & Tierney, 2020), and NeuRewriter (Chen & Tian, 2019), which employ transformer structures to iteratively refine solutions, yielding superior routing outcomes.

Tables 1 and 2 report the routing performance between our proposed SAT and baseline methods on uniform-distribution and cross-distribution scenarios, respectively. The main results are summarized as follows.

Table 1

Comparison results with baseline methods on TSP20-100 and CVRP20-100 with a uniform node distribution.

|      | Method             | Type             | N = 20      |              |      | N = 50       |              |           | N = 100      |              |           |
|------|--------------------|------------------|-------------|--------------|------|--------------|--------------|-----------|--------------|--------------|-----------|
|      |                    |                  | Length      | Gap          | Time | Length       | Gap          | Time      | Length       | Gap          | Time      |
| TSP  | Concorde           | Exact Solver     | 3.83        | 0.00%        | 5 m  | 5.69         | 0.00%        | 13 m      | 7.76         | 0.00%        | 1h        |
|      | LKH                | Heuristic Solver | 3.83        | 0.00%        | 43s  | 5.69         | 0.00%        | 6 m       | 7.76         | 0.00%        | 26 m      |
|      | Gurobi             | Heuristic Solver | 3.83        | 0.00%        | 9s   | 5.69         | 0.00%        | 2 m       | 7.76         | 0.00%        | 18 m      |
|      | OR Tools           | Heuristic Solver | 3.86        | 0.78%        | 1 m  | 5.85         | 2.81%        | 5 m       | 8.07         | 3.99%        | 24 m      |
|      | Nearest Neighbor   | Non-learned      | 4.50        | 17.49%       | 1s   | 7.00         | 23.02%       | 1s        | 9.68         | 24.74%       | 1s        |
|      | Nearest Insertion  | Non-learned      | 4.33        | 13.05%       | 1s   | 6.78         | 19.16%       | 2s        | 9.46         | 21.91%       | 6s        |
|      | Random Insertion   | Non-learned      | 4.00        | 4.44%        | 1s   | 6.13         | 7.73%        | 2s        | 8.52         | 9.79%        | 3s        |
|      | Farthest Insertion | Non-learned      | 3.93        | 2.61%        | 1s   | 6.01         | 5.62%        | 2s        | 8.35         | 7.60%        | 7s        |
|      | SA                 | Heuristic        | 3.90        | 1.83%        | 1 m  | 5.89         | 3.51%        | 3 m       | 8.09         | 4.25%        | 18 m      |
|      | GA                 | Heuristic        | 3.86        | 0.78%        | 1 m  | 5.83         | 2.46%        | 5 m       | 8.01         | 3.22%        | 23 m      |
|      | ACO                | Heuristic        | 3.85        | 0.52%        | 1 m  | 5.79         | 1.76%        | 4 m       | 7.95         | 2.45%        | 21 m      |
|      | AM-greedy          | Construction DRL | 3.85        | 0.52%        | 1s   | 5.80         | 1.93%        | 2s        | 8.10         | 4.38%        | 6s        |
|      | AM-sampling        | Construction DRL | 3.84        | 0.26%        | 14 m | 5.72         | 0.53%        | 47 m      | 7.94         | 2.32%        | 1.5h      |
|      | POMO               | Construction DRL | 3.84        | 0.26%        | 1s   | 5.72         | 0.53%        | 3s        | 7.85         | 1.16%        | 12s       |
|      | POMO ×8 augment    | Construction DRL | <b>3.83</b> | <b>0.00%</b> | 4 m  | 5.70         | 0.18%        | 21 m      | 7.80         | 0.52%        | 1h        |
|      | Wu et al.          | Improvement DRL  | <b>3.83</b> | <b>0.00%</b> | 12 m | 5.74         | 0.88%        | 17 m      | 8.02         | 3.35%        | 25 m      |
|      | DACT               | Improvement DRL  | <b>3.83</b> | <b>0.00%</b> | 2 m  | 5.72         | 0.53%        | 7 m       | 7.89         | 1.68%        | 20 m      |
|      | Neural-2-Opt       | Improvement DRL  | <b>3.83</b> | <b>0.00%</b> | 15 m | 5.71         | 0.35%        | 29 m      | 7.85         | 1.16%        | 41 m      |
|      | Ours               | Improvement DRL  | <b>3.83</b> | <b>0.00%</b> | 2 m  | <b>5.69</b>  | <b>0.00%</b> | 6 m       | <b>7.81</b>  | <b>0.64%</b> | 16 m      |
| CVRP | LKH                | Heuristic Solver | 6.10        | 0.00%        | 2h   | 10.38        | 0.00%        | 7h        | 15.65        | 0.00%        | 14h       |
|      | Gurobi             | Heuristic Solver | 6.10        | 0.00%        | 1 m  | <i>OB</i>    | <i>OB</i>    | <i>OB</i> | <i>OB</i>    | <i>OB</i>    | <i>OB</i> |
|      | OR Tools           | Heuristic Solver | 6.46        | 5.90%        | 2 m  | 11.28        | 8.67%        | 14 m      | 17.11        | 9.33%        | 48 m      |
|      | SA                 | Heuristic        | 6.45        | 5.74%        | 2 m  | 11.18        | 7.71%        | 10 m      | 16.94        | 8.24%        | 36 m      |
|      | GA                 | Heuristic        | 6.39        | 4.75%        | 2 m  | 11.09        | 6.84%        | 13 m      | 16.83        | 7.54%        | 45 m      |
|      | ACO                | Heuristic        | 6.31        | 3.44%        | 2 m  | 10.91        | 5.11%        | 12 m      | 16.56        | 5.82%        | 42 m      |
|      | AM-greedy          | Construction DRL | 6.40        | 4.92%        | 1s   | 10.94        | 5.39%        | 3s        | 16.75        | 7.03%        | 8s        |
|      | AM-sampling        | Construction DRL | 6.25        | 2.46%        | 16 m | 10.63        | 2.41%        | 56 m      | 16.24        | 3.77%        | 2h        |
|      | POMO               | Construction DRL | 6.35        | 4.10%        | 1s   | 10.73        | 3.37%        | 4s        | 16.12        | 3.00%        | 21s       |
|      | POMO ×8 augment    | Construction DRL | 6.14        | 0.66%        | 5 m  | 10.45        | 0.67%        | 28 m      | 15.77        | 0.77%        | 1.5h      |
|      | NLNS               | Improvement DRL  | 6.19        | 1.48%        | 7 m  | 10.56        | 1.73%        | 24 m      | 16.08        | 2.75%        | 1h        |
|      | NeuRewriter        | Improvement DRL  | 6.16        | 0.98%        | 22 m | 10.51        | 1.25%        | 35 m      | 16.10        | 2.88%        | 1h        |
|      | Wu et al.          | Improvement DRL  | 6.16        | 0.98%        | 23 m | 10.68        | 2.89%        | 48 m      | 16.29        | 4.09%        | 1h        |
|      | DACT               | Improvement DRL  | 6.14        | 0.66%        | 4 m  | 10.55        | 1.64%        | 16 m      | 16.20        | 3.51%        | 41 m      |
|      | Ours               | Improvement DRL  | <b>6.12</b> | <b>0.33%</b> | 2 m  | <b>10.43</b> | <b>0.48%</b> | 8 m       | <b>15.77</b> | <b>0.77%</b> | 18 m      |

Bold results indicate those outperforming DRL-based baselines. AM with sampling rollout samples 1,280 solutions. The notation *OB* means out of budget.

Table 2

Comparison results with DRL-based methods on TSPs and CVRPs with diverse node distributions.

| Distribution<br>&Method |         | TSP50       |              |             |              |      | TSP100      |              |             |              |      | CVRP50       |              |              |              |      | CVRP100      |              |              |              |      |
|-------------------------|---------|-------------|--------------|-------------|--------------|------|-------------|--------------|-------------|--------------|------|--------------|--------------|--------------|--------------|------|--------------|--------------|--------------|--------------|------|
|                         |         | Length      | Gap          | Length*     | Gap*         | Time | Length      | Gap          | Length*     | Gap*         | Time | Length       | Gap          | Length*      | Gap*         | Time | Length       | Gap          | Length*      | Gap*         | Time |
| Expansion               | Solvers | 6.13        | 0.00%        | 6.20        | 0.00%        | 12 m | 8.20        | 0.00%        | 9.28        | 0.00%        | 1h   | 11.87        | 0.00%        | 12.15        | 0.00%        | 5h   | 16.99        | 0.00%        | 17.39        | 0.00%        | 9h   |
|                         | AM      | 6.36        | 3.75%        | 6.54        | 5.48%        | 7 m  | 8.45        | 3.05%        | 9.70        | 4.53%        | 27 m | 12.22        | 2.95%        | 12.69        | 4.44%        | 11 m | 17.65        | 3.88%        | 18.22        | 4.77%        | 42 m |
|                         | POMO    | 6.30        | 2.77%        | 6.62        | 6.77%        | 4 m  | 8.41        | 2.56%        | 9.52        | 2.59%        | 20 m | 12.08        | 1.77%        | 12.50        | 2.88%        | 6 m  | 17.56        | 3.35%        | 18.05        | 3.80%        | 32 m |
|                         | DACT    | 6.31        | 2.94%        | 6.58        | 6.13%        | 2 m  | 8.42        | 2.68%        | 9.59        | 3.34%        | 7 m  | 12.13        | 2.19%        | 12.56        | 3.37%        | 4 m  | 17.59        | 3.53%        | 18.11        | 4.14%        | 15 m |
|                         | Ours    | <b>6.19</b> | <b>0.98%</b> | <b>6.27</b> | <b>1.13%</b> | 2 m  | <b>8.34</b> | <b>1.71%</b> | <b>9.39</b> | <b>1.19%</b> | 6 m  | <b>11.96</b> | <b>0.76%</b> | <b>12.25</b> | <b>0.82%</b> | 2 m  | <b>17.33</b> | <b>2.00%</b> | <b>17.75</b> | <b>2.07%</b> | 7 m  |
| Implosion               | Solvers | 6.41        | 0.00%        | 6.74        | 0.00%        | 14 m | 8.77        | 0.00%        | 9.51        | 0.00%        | 1h   | 12.12        | 0.00%        | 12.47        | 0.00%        | 5h   | 17.27        | 0.00%        | 17.63        | 0.00%        | 10h  |
|                         | AM      | 6.63        | 3.43%        | 7.22        | 7.12%        | 8 m  | 9.11        | 3.88%        | 10.19       | 7.15%        | 31 m | 12.56        | 3.63%        | 12.92        | 3.61%        | 12 m | 17.69        | 2.43%        | 18.40        | 4.37%        | 49 m |
|                         | POMO    | 6.58        | 2.65%        | 7.18        | 6.53%        | 4 m  | 8.91        | 1.60%        | 9.83        | 3.36%        | 21 m | 12.44        | 2.64%        | 12.72        | 2.00%        | 6 m  | 17.67        | 2.32%        | 18.27        | 3.63%        | 37 m |
|                         | DACT    | 6.59        | 2.81%        | 7.18        | 6.53%        | 2 m  | 8.96        | 2.17%        | 9.94        | 4.52%        | 8 m  | 12.47        | 2.89%        | 12.78        | 2.49%        | 4 m  | 17.67        | 2.32%        | 18.31        | 3.86%        | 17 m |
|                         | Ours    | <b>6.45</b> | <b>0.62%</b> | <b>6.81</b> | <b>1.04%</b> | 2 m  | <b>8.85</b> | <b>0.91%</b> | <b>9.65</b> | <b>1.47%</b> | 6 m  | <b>12.27</b> | <b>1.24%</b> | <b>12.55</b> | <b>0.64%</b> | 2 m  | <b>17.58</b> | <b>1.80%</b> | <b>18.06</b> | <b>2.44%</b> | 7 m  |
| Explosion               | Solvers | 6.34        | 0.00%        | 6.70        | 0.00%        | 14 m | 8.59        | 0.00%        | 9.15        | 0.00%        | 1h   | 12.03        | 0.00%        | 12.31        | 0.00%        | 5h   | 17.19        | 0.00%        | 17.55        | 0.00%        | 10h  |
|                         | AM      | 6.60        | 4.10%        | 7.10        | 5.97%        | 8 m  | 9.01        | 4.89%        | 9.95        | 8.74%        | 29 m | 12.30        | 2.24%        | 12.83        | 4.22%        | 13 m | 17.78        | 3.43%        | 18.34        | 4.50%        | 49 m |
|                         | POMO    | 6.53        | 3.00%        | 7.06        | 5.37%        | 4 m  | 8.85        | 3.03%        | 9.65        | 5.46%        | 19 m | 12.26        | 1.08%        | 12.70        | 3.17%        | 7 m  | 17.65        | 2.68%        | 18.15        | 3.42%        | 37 m |
|                         | DACT    | 6.55        | 3.31%        | 7.07        | 5.52%        | 2 m  | 8.90        | 3.61%        | 9.74        | 6.45%        | 6 m  | 12.28        | 2.08%        | 12.74        | 3.49%        | 4 m  | 17.69        | 2.91%        | 18.21        | 3.76%        | 17 m |
|                         | Ours    | <b>6.44</b> | <b>1.58%</b> | <b>6.77</b> | <b>1.04%</b> | 2 m  | <b>8.75</b> | <b>1.86%</b> | <b>9.45</b> | <b>3.28%</b> | 5 m  | <b>12.19</b> | <b>1.33%</b> | <b>12.51</b> | <b>1.62%</b> | 2 m  | <b>17.55</b> | <b>2.09%</b> | <b>17.97</b> | <b>2.39%</b> | 8 m  |
| Grid                    | Solvers | 6.02        | 0.00%        | 6.29        | 0.00%        | 10 m | 7.86        | 0.00%        | 7.99        | 0.00%        | 1h   | 11.73        | 0.00%        | 11.98        | 0.00%        | 5h   | 16.72        | 0.00%        | 17.07        | 0.00%        | 9h   |
|                         | AM      | 6.24        | 3.65%        | 6.64        | 5.56%        | 7 m  | 8.12        | 3.31%        | 8.49        | 6.26%        | 27 m | 12.13        | 3.41%        | 12.52        | 4.51%        | 10 m | 17.31        | 3.53%        | 17.89        | 4.80%        | 41 m |
|                         | POMO    | 6.21        | 3.16%        | 6.42        | 2.07%        | 4 m  | 8.02        | 2.04%        | 8.34        | 4.38%        | 18 m | 11.92        | 1.62%        | 12.36        | 3.17%        | 5 m  | 17.22        | 2.99%        | 17.76        | 4.04%        | 31 m |
|                         | DACT    | 6.22        | 3.32%        | 6.49        | 3.18%        | 2 m  | 8.06        | 2.54%        | 8.39        | 5.01%        | 6 m  | 11.99        | 2.22%        | 12.41        | 3.59%        | 3 m  | 17.25        | 3.17%        | 17.82        | 4.39%        | 14 m |
|                         | Ours    | <b>6.08</b> | <b>1.00%</b> | <b>6.33</b> | <b>0.64%</b> | 2 m  | <b>7.98</b> | <b>1.53%</b> | <b>8.18</b> | <b>2.38%</b> | 5 m  | <b>11.85</b> | <b>1.02%</b> | <b>12.13</b> | <b>1.25%</b> | 2 m  | <b>17.08</b> | <b>2.15%</b> | <b>17.51</b> | <b>2.58%</b> | 6 m  |
| Cluster                 | Solvers | 5.89        | 0.00%        | 6.17        | 0.00%        | 10 m | 7.45        | 0.00%        | 7.62        | 0.00%        | 1h   | 11.69        | 0.00%        | 11.85        | 0.00%        | 5h   | 16.71        | 0.00%        | 17.00        | 0.00%        | 8h   |
|                         | AM      | 6.11        | 3.74%        | 6.66        | 7.94%        | 6 m  | 7.80        | 4.70%        | 7.95        | 4.33%        | 25 m | 12.04        | 2.99%        | 12.46        | 5.15%        | 10 m | 17.29        | 3.47%        | 17.95        | 5.59%        | 38 m |
|                         | POMO    | 6.08        | 3.23%        | 6.35        | 2.92%        | 3 m  | 7.70        | 3.36%        | 7.83        | 2.76%        | 17 m | 11.89        | 1.71%        | 12.33        | 4.05%        | 5 m  | 17.24        | 3.17%        | 17.75        | 4.41%        | 29 m |
|                         | DACT    | 6.09        | 3.40%        | 6.44        | 4.38%        | 1 m  | 7.72        | 3.62%        | 7.86        | 3.15%        | 6 m  | 11.93        | 2.05%        | 12.37        | 4.39%        | 3 m  | 17.25        | 3.23%        | 17.81        | 4.76%        | 13 m |
|                         | Ours    | <b>5.92</b> | <b>0.51%</b> | <b>6.26</b> | <b>1.46%</b> | 1 m  | <b>7.65</b> | <b>2.68%</b> | <b>7.79</b> | <b>2.23%</b> | 5 m  | <b>11.84</b> | <b>1.28%</b> | <b>12.08</b> | <b>1.94%</b> | 1 m  | <b>17.08</b> | <b>2.21%</b> | <b>17.49</b> | <b>2.88%</b> | 6 m  |

The length, gap, and time are computed over 11,000 instances, which involves 10,000 typical instances and 1,000 atypical instances. The superscript \* represents the average results over 1,000 atypical instances.

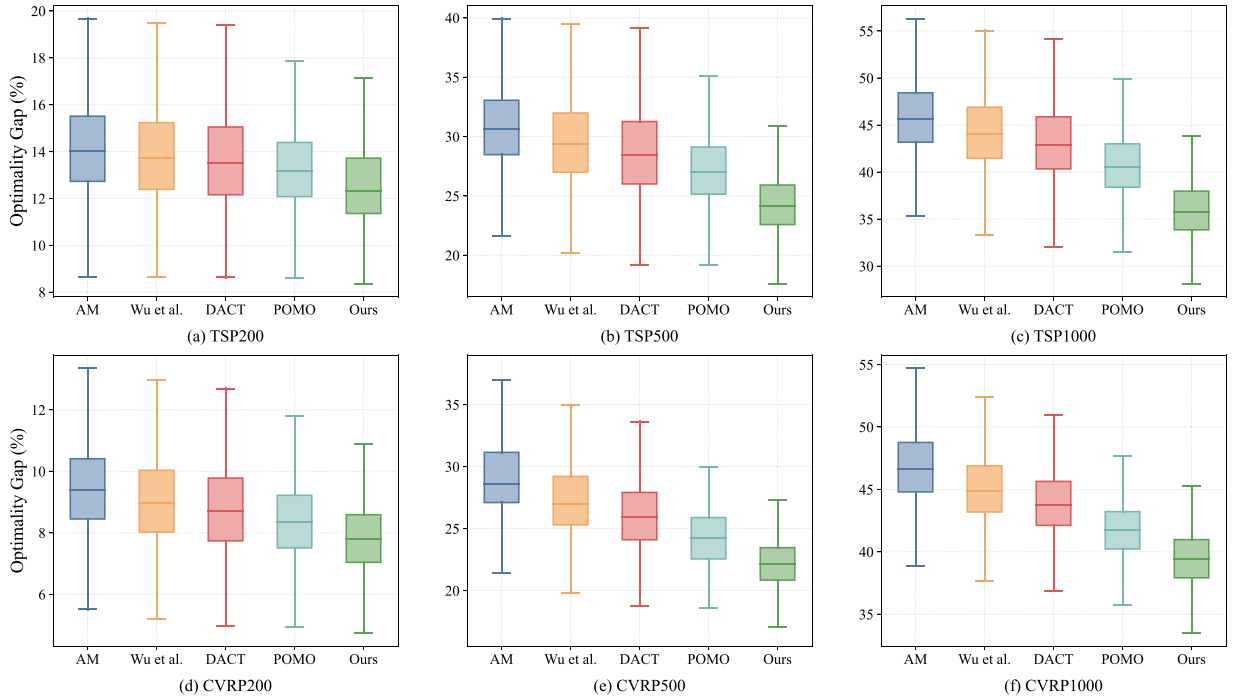


Fig. 5. Generalization results on larger-scale TSPs and CVRPs with 200, 500, and 1,000 nodes following the uniform nodal distribution.

- In both uniform-distribution and cross-distribution paradigms, our SAT consistently outperforms heuristic and DRL-based methods by a substantial margin across TSPs and CVRPs, regardless of problem sizes. This overall enhancement underscores the ability of our SAT to strike a delicate balance between solution quality and computational efficiency. The superiority of our approach becomes particularly evident in addressing large-scale routing challenges (i.e. TSP100 and CVRP100), further highlighting the efficacy of the innovative MHSA mechanism.
- Within the realm of uniform-distribution scenarios, our proposed approach stands out by achieving the most favorable path lengths compared to both construction and improvement DRL methods, as delineated in Table 1. A striking observation stems from the exceptional routing proficiency of our SAT on the large-scale TSP100, where it achieves an average path length of 7.81 with an optimality gap of 0.64%. In contrast, it surpasses construction-type AM, POMO, and improvement-type Wu et al. and DACT by demonstrating a reduction of 3.74%, 0.52%, 2.71%, and 1.04% in optimality gap, respectively. Analogously, our methodology yields optimality gaps of 0.33%, 0.48%, and 0.77% on CVRP20, CVRP50, and CVRP100, respectively, which outstrips state-of-the-art DACT by 0.33%, 1.16%, and 2.74% correspondingly. Furthermore, a notable finding is the increasingly pronounced advantage of our algorithm in terms of both solution accuracy and speed as the problem size escalates.
- Regarding cross-distribution routing scenarios, we present comprehensive results across the entire test set, alongside the average path length and optimality gap on 1,000 atypical instances to indicate the worst-case performance, as depicted in Table 2. It is evident that our approach exhibits superior performance compared to prevailing DRL methods across five pairs of cross-distribution groups. Take TSP100 as an example, SAT delivers overall optimality gaps of 1.19%, 1.47%, 3.28%, 2.38%, and 2.23% on five pairs of distributions, outperforming the second-best method POMO by margins of 2.15%, 3.05%, 3.17%, 2.63%, and 0.92%, respectively. When focusing on worst-case performance, the efficacy of our method becomes particularly prominent for atypical instances from the minority group. Notably, SAT

achieve an average gap of 1.47% on atypical instances with the implosion distribution in TSP100, surpassing DRL methods AM, POMO, and DACT by 5.68%, 1.89%, and 3.05%, respectively. This accomplishment convincingly demonstrates that our SAT learns the most essential route optimization strategies through the synergetic attention mechanism, thereby resulting in outstanding and resilient outcomes across instances with diverse distributions.

#### 4.3. Generalization analysis

To assess the generalization performance of the proposed approach, we deploy our model across two distinct scenarios: (1) larger-scale instances featuring a uniform nodal distribution; and (2) similar-scale instances but with different distributions as the training set, sourced from real-world benchmarks TSPLIB and CVRPLIB.

##### 4.3.1. Generalize on larger-scale instances

Directly training DRL models from scratch to tackle ultra-large-scale routing problems, such as TSP1,000 and CVRP1,000, is both intractable and impractical. Therefore, we endeavor to evaluate the generalization ability of our SAT, which is essential of applying learned policies to tackle unseen and challenging problems. To this end, we train our model on 10,000 instances with a uniform distribution in TSP100. Subsequently, we employ the trained model to address larger-scale problems, namely TSP200, TSP500, and TSP1,000, each comprising 1,000 instances with the same uniform distribution. We conduct 10 independent replications to ensure stable and representative indicators. In addition, we generalize the model trained on CVRP100 to tackle larger-scale CVRP200, CVRP500, and CVRP1,000 using a similar paradigm.

Fig. 5 illustrates the generalization performance between our model and mainstream DRL methods. Overall, our SAT exhibits the best optimality gaps and interquartile ranges on both TSPs and CVRPs, outperforming other DRL methods by a considerable margin. For instance, focusing on TSP500, our SAT achieves a median gap of 24.17%, surpassing AM, Wu et al. DACT, and POMO by reductions of 6.48%, 5.18%, 4.31%, and 2.86%, respectively. Regarding CVRP500, the lower and

**Table 3**  
Generalization results on TSPLib.

| Instance       | Optimal | AM      | POMO    | Wu et al. | DACT    | Ours           |
|----------------|---------|---------|---------|-----------|---------|----------------|
| eil51          | 426     | 439     | 436     | 438       | 437     | <b>434</b>     |
| berlin52       | 7,542   | 8,352   | 7,836   | 8,168     | 8,045   | <b>7,816</b>   |
| st70           | 675     | 691     | 684     | 689       | 687     | <b>683</b>     |
| pr76           | 108,159 | 111,605 | 112,498 | 111,798   | 111,597 | <b>111,237</b> |
| rat99          | 1,211   | 1,340   | 1,286   | 1,326     | 1,310   | <b>1,258</b>   |
| rd100          | 7,910   | 8,252   | 8,180   | 8,227     | 8,218   | <b>8,142</b>   |
| lin105         | 14,379  | 15,148  | 14,922  | 15,073    | 15,027  | <b>14,795</b>  |
| pr107          | 44,303  | 53,846  | 52,835  | 53,479    | 53,236  | <b>48,391</b>  |
| ch130          | 6,110   | 6,324   | 6,279   | 6,310     | 6,297   | <b>6,258</b>   |
| pr136          | 96,772  | 104,194 | 102,646 | 103,642   | 103,162 | <b>101,867</b> |
| ch150          | 6,528   | 6,930   | 6,844   | 6,899     | 6,872   | <b>6,774</b>   |
| pr152          | 73,682  | 82,001  | 80,151  | 81,352    | 80,896  | <b>79,105</b>  |
| u159           | 42,080  | 49,477  | 47,436  | 48,767    | 48,219  | <b>46,183</b>  |
| rat195         | 2,323   | 2,612   | 2,554   | 2,591     | 2,576   | <b>2,466</b>   |
| kroA200        | 29,368  | 35,637  | 34,972  | 35,376    | 35,241  | <b>32,318</b>  |
| Gap [50, 100]  | 0.00%   | 6.00%   | 3.56%   | 5.21%     | 4.48%   | <b>2.68%</b>   |
| Gap [100, 150] | 0.00%   | 8.48%   | 7.06%   | 7.98%     | 7.65%   | <b>4.55%</b>   |
| Gap [150, 200] | 0.00%   | 13.76%  | 11.27%  | 12.80%    | 12.11%  | <b>7.42%</b>   |
| Avg. Gap       | 0.00%   | 9.41%   | 7.30%   | 8.66%     | 8.08%   | <b>4.88%</b>   |

Gap [50, 100] represents the average gap of instances whose number of nodes ranges from 50 to 100, and the other forms are the same. Bold results indicate the best performance across both construction and improvement-type DRL methods.

**Table 4**  
Generalization results on CVRPLib.

| Instance       | Optimal | AM     | POMO   | Wu et al. | DACT   | Ours          |
|----------------|---------|--------|--------|-----------|--------|---------------|
| X-n101-k25     | 27,591  | 38,264 | 29,484 | 35,106    | 32,566 | <b>29,138</b> |
| X-n106-k14     | 26,362  | 27,923 | 27,762 | 27,459    | 27,861 | <b>27,381</b> |
| X-n110-k13     | 14,971  | 16,320 | 15,896 | 16,176    | 16,182 | <b>15,696</b> |
| X-n115-k10     | 12,747  | 14,055 | 13,592 | 14,054    | 13,986 | <b>13,702</b> |
| X-n120-k6      | 13,332  | 14,456 | 14,351 | 14,403    | 16,379 | <b>14,107</b> |
| X-n125-k30     | 55,539  | 74,329 | 69,560 | 72,677    | 72,038 | <b>62,303</b> |
| X-n129-k18     | 28,940  | 30,869 | 30,155 | 30,625    | 30,427 | <b>30,172</b> |
| X-n134-k13     | 10,916  | 13,592 | 13,483 | 13,792    | 13,677 | <b>13,067</b> |
| X-n139-k10     | 13,590  | 14,893 | 14,132 | 14,621    | 14,390 | <b>14,103</b> |
| X-n143-k7      | 15,700  | 18,251 | 17,923 | 18,139    | 18,019 | <b>17,586</b> |
| X-n153-k22     | 21,220  | 38,423 | 26,386 | 34,070    | 30,453 | <b>24,492</b> |
| X-n157-k13     | 16,876  | 22,051 | 19,978 | 21,327    | 20,696 | <b>18,834</b> |
| X-n181-k23     | 25,569  | 27,826 | 27,428 | 27,686    | 27,408 | <b>27,218</b> |
| X-n190-k8      | 16,980  | 37,820 | 22,310 | 32,234    | 27,384 | <b>20,526</b> |
| X-n200-k36     | 58,578  | 76,528 | 73,135 | 75,325    | 74,385 | <b>68,687</b> |
| Gap (100, 120] | 0.00%   | 14.46% | 7.09%  | 11.55%    | 12.88% | <b>5.65%</b>  |
| Gap (120, 150] | 0.00%   | 18.83% | 14.22% | 17.23%    | 16.16% | <b>10.39%</b> |
| Gap (150, 200] | 0.00%   | 54.79% | 21.25% | 42.73%    | 32.32% | <b>14.32%</b> |
| Avg. Gap       | 0.00%   | 29.36% | 14.19% | 23.83%    | 20.45% | <b>10.08%</b> |

Gap (100, 120] represents the average gap of instances whose number of nodes ranges from 100 to 120, and the other forms are the same. Bold results indicate the best performance across both construction and improvement-type DRL methods.

upper quartiles of SAT are 20.84% and 23.46%, which are 1.71% and 2.43% lower than those of the second-best method POMO. This substantial enhancement underscores the superior scalability and generalizability of our approach, thereby empowering its efficacy in addressing larger-scale in-distribution problems in real-life applications.

#### 4.3.2. Generalize on real-world benchmarks

In the context of practical implementation and advancement, it is imperative to deploy trained models to address instances of similar scale but divergent in distribution, which are frequently encountered in logistics and transportation domains. To assess this crucial generalization ability, we evaluate the proposed approach on mainstream real-world benchmarks TSPLIB and CVRPLIB. Adhering to established protocols in Jiang et al. (2022), the training set for DRL models consists of 15,000 instances, of which 10,000 represent typical instances conforming to a uniform distribution, while the remaining five groups

**Table 5**  
Comparison gaps of different operators in our method on TSP20-100 and CVRP20-100.

| Operator | TSP          |              |              | CVRP         |              |              |
|----------|--------------|--------------|--------------|--------------|--------------|--------------|
|          | 20           | 50           | 100          | 20           | 50           | 100          |
| Swap     | 0.26%        | 0.35%        | 1.03%        | 0.66%        | 0.96%        | 1.28%        |
| Relocate | <b>0.00%</b> | 0.18%        | 0.90%        | 0.49%        | 0.77%        | 1.02%        |
| 2-opt    | <b>0.00%</b> | <b>0.00%</b> | <b>0.64%</b> | <b>0.33%</b> | <b>0.48%</b> | <b>0.77%</b> |

of 1000 atypical instances each adhere to distributions including expansion, implosion, explosion, grid, and cluster, respectively. Building upon this framework, we apply the model trained on TSP100 and CVRP100 to address instances whose number of nodes ranges from 50 to 200. The mean tour lengths, computed across 10 independent replications for TSP and CVRP instances, are presented in Tables 3 and 4.

Overall, our proposed approach outperforms other state-of-the-art methods and achieves results closest to optimality across both TSPs and CVRPs. Concerning TSPLIB, our SAT achieves an average gap of 4.88%, outstripping DRL methods AM, POMO, Wu et al. and DACT by reductions of 4.53%, 2.42%, 7.92%, and 7.23%, respectively. With regards to CVRPs, the optimality gaps of our methodology on instances with 100–120, 120–150, and 150–200 nodes are 5.65%, 10.39%, and 14.32%, respectively, outperforming the sophisticated DRL method POMO by 1.44%, 3.83%, and 6.93%, respectively. It is noteworthy that the superiority of our SAT becomes more pronounced when tackling more challenging problems in comparison with prevailing DRL models. This accomplishment underscores the remarkable efficacy of SAT when confronted with real-world complicated and changeable scenarios.

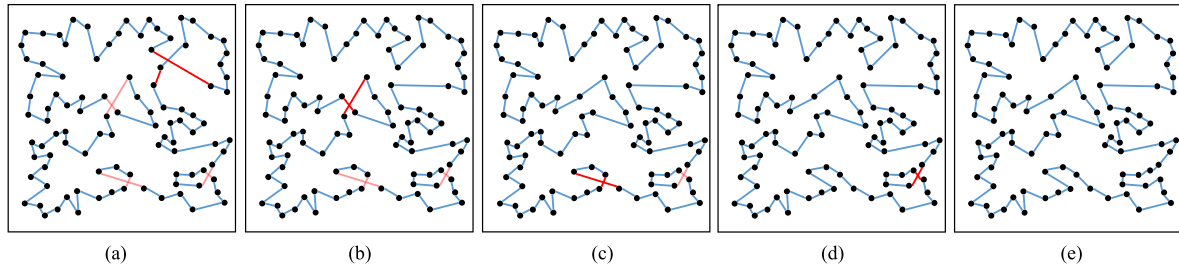
#### 4.4. Ablation study

##### 4.4.1. Effect of different operators

The proposed approach leverages an improvement-type DRL paradigm, which progressively refines the existing solution by employing node-pair operators such as swap, relocate, and 2-opt. Recognizing the pivotal role of selecting the most effective operator for route optimization, Table 5 presents a comprehensive analysis of the optimality gaps achieved by our approach with various operators. Remarkably, our SAT, when utilizing the 2-opt operator, exhibits the most favorable routing performance across both TSPs and CVRPs. Furthermore, our approach consistently outperforms the state-of-the-art improvement-type method, DACT, by a minimum of 2.23% on CVRP100, regardless of the specific node-pair operator employed. This notable accomplishment underscores the robustness and superiority of our meticulously devised approach.

##### 4.4.2. Effect of different losses

Our proposed SAT introduces a novel approach that incorporates multiple decoders and designs two innovative regularization losses  $\mathcal{L}_{\text{INT}}$  and  $\mathcal{L}_{\text{EXT}}$  into the backbone reinforcement term  $J_{\text{RL}}$ . This augmentation aims to expand the search space, with the ultimate goal of achieving superior routing performance. Table 6 provides a comprehensive analysis of the impact of gradually introducing these two losses on both TSPs and CVRPs. Initially, we establish a vanilla framework using the actor-critic algorithm, which yields an average optimality gap of 6.13% on CVRP100. Subsequently, the integration of our proposed  $\mathcal{L}_{\text{INT}}$  and  $\mathcal{L}_{\text{EXT}}$  into the overall loss function enables the model to achieve enhanced optimality gaps of 3.83% and 3.32%, contributing to significant relative gains by 2.30% and 2.81%, respectively. Finally, when both losses are combined, the resulting model exhibits remarkable routing performance, achieving an average gap of 0.77% on CVRP100. This outstanding achievement convincingly demonstrates the effectiveness of our well-designed loss function and its intrinsic diversity mechanism.



**Fig. 6.** Visualization of learned policies on TSP100 as an example. From (a) to (e), our approach iteratively refines the existing route via 2-opt operators, thereby resulting in achieving the optimal solution. More specifically, dark red segment pairs indicate that they require to be adjusted by 2-opt at the current time, and light red segment pairs represent those with potential for future improvement. Constitutionally, our SAT learns improvement-type DRL to solve segment-crossing problems with 2-opt node-pair operators, leading to a satisfactory route with no intersection.

**Table 6**

Comparison gaps of different losses in our method on TSP20-100 and CVRP20-100.

| Components of Losses |                     |                     | TSP   |       |       | CVRP  |       |       |
|----------------------|---------------------|---------------------|-------|-------|-------|-------|-------|-------|
| $J_{RL}$             | $\mathcal{L}_{INT}$ | $\mathcal{L}_{EXT}$ | 20    | 50    | 100   | 20    | 50    | 100   |
| ✓                    |                     |                     | 0.52% | 1.76% | 3.87% | 4.26% | 4.82% | 6.13% |
| ✓                    | ✓                   |                     | 0.26% | 0.88% | 2.06% | 0.98% | 2.02% | 3.83% |
| ✓                    |                     | ✓                   | 0.00% | 0.53% | 1.41% | 0.66% | 1.45% | 3.32% |
| ✓                    | ✓                   | ✓                   | 0.00% | 0.00% | 0.64% | 0.33% | 0.48% | 0.77% |

#### 4.5. Visualization

We attempt to provide a vivid illustration of the impact of learned policies based on the 2-opt operator during the route optimization process. Utilizing an instance from TSP100 as a case study, we visually depict the evolution of the route overlapping changes in Fig. 6. Concretely, our model learns to select a pair of nodes iteratively and leverages 2-opt to reconfigure the route by reversing the sequence of access between the two chosen nodes. Through this approach, it can be observed that our SAT effectively resolves the segment-crossing issues, depicted by dark red lines, ultimately leading to the attainment of an optimal solution devoid of intersections. This achievement unequivocally underscores the effectiveness and practicality of our improvement-type DRL approach integrated with 2-opt in addressing a variety of routing problems.

#### 5. Conclusion

This study is centered on advancing improvement-type DRL for addressing routing problems. We identify the circularity and symmetry inherent in routing solutions and develop a synergetic attention-driven transformer to capture and characterize these features. Specifically, the encoder processes node coordinate and access sequence features through a dual pathway to generate high-quality representative tokens. The decoder leverages a compatibility block to select a node pair for refining the solution via 2-opt. Moreover, to promote diversity, we introduce multiple decoders and incorporate two regularization losses alongside the reinforcement term to expand the search space, thereby effectively enhancing routing performance. Comprehensive experiments demonstrate that our SAT outperforms both construction-type and improvement-type DRL methods by a substantial margin in both inference and generalization tasks. In future endeavors, we aim to explore hierarchical DRL models, which are capable of selecting both the operator and the corresponding node pair within the improvement-type routing paradigm. Additionally, we aspire to apply our approach to address other COPs where circularity plays a pivotal role, such as DNA encoding in bioinformatics.

#### CRedit authorship contribution statement

**Qingshu Guan:** Methodology, Software, Writing – original draft. **Hui Cao:** Conceptualization, Methodology, Supervision, Project administration, Writing – review & editing. **Lixin Jia:** Investigation, Software, Validation. **Dapeng Yan:** Resources, Methodology, Writing – review & editing. **Badong Chen:** Formal analysis, Data curation, Validation.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Acknowledgments

This work was funded by the National Key Research and Development Program of China (No. 2019YFB1312000), the National Natural Science Foundation of China (No. U21A20485, 62306232, 62436005, and 62088102), the Natural Science Basic Research Program of Shaanxi Province (No. 2023-JC-QN-0662), and the State Key Laboratory of Electrical Insulation and Power Equipment of China (No. EIPE23416).

#### Data availability

Data will be made available on request.

#### References

- Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2020). Concorde TSP solver. URL <http://www.math.uwaterloo.ca/tsp/concorde/>.
- Bello, I., Pham, H., Le, Q. V., Norouzi, M., & Bengio, S. (2017). Neural combinatorial optimization with reinforcement learning. In *International conference on learning representations*.
- Bogrybayeva, A., Meraliyev, M., Mustakhov, T., & Dauletbayev, B. (2024). Machine learning to solve vehicle routing problems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 1–19.
- Bogrybayeva, A., Yoon, T., Ko, H., Lim, S., Yun, H., & Kwon, C. (2023). A deep reinforcement learning approach for solving the traveling salesman problem with drone. *Transportation Research Part C: Emerging Technologies*, 148, Article 103981.
- Chen, X., & Tian, Y. (2019). Learning to perform local rewriting for combinatorial optimization. *Advances in Neural Information Processing Systems*, 32.
- d O Costa, P. R., Rhuggenaath, J., Zhang, Y., & Akcay, A. (2020). Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In *Asian conference on machine learning* (pp. 465–480). PMLR.
- Fan, M., Wu, Y., Liao, T., Cao, Z., Guo, H., Sartoretto, G., et al. (2022). Deep reinforcement learning for UAV routing in the presence of multiple charging stations. *IEEE Transactions on Vehicular Technology*, 72(5), 5732–5746.
- Gmira, M., Gendreau, M., Lodi, A., & Potvin, J. Y. (2021). Tabu search for the time-dependent vehicle routing problem with time windows on a road network. *European Journal of Operational Research*, 288(1), 129–140.
- Guan, Q., Hong, X., Ke, W., Zhang, L., Sun, G., & Gong, Y. (2021). Kohonen self-organizing map based route planning: A revisit. In *2021 IEEE/RSJ international conference on intelligent robots and systems* (pp. 7969–7976). IEEE.

- Helsgaun, K. (2017). *An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems* (pp. 24–50). Roskilde: Roskilde University.
- Hlaing, Z. C. S. S., & Khine, M. A. (2011). Solving traveling salesman problem by using improved ant colony optimization algorithm. *International Journal of Information and Education Technology*, 1(5), 404.
- Hottung, A., & Tierney, K. (2020). Neural large neighborhood search for the capacitated vehicle routing problem. In *ECAI 2020* (pp. 443–450). IOS Press.
- Hussain, A., Muhammad, Y. S., Nauman Sajid, M., Hussain, I., Mohamd Shoukry, A., & Gani, S. (2017). Genetic algorithm for traveling salesman problem with modified cycle crossover operator. *Computational Intelligence and Neuroscience*, 2017.
- Jiang, Y., Cao, Z., Wu, Y., Song, W., & Zhang, J. (2023). Ensemble-based deep reinforcement learning for vehicle routing problems under distribution shift. *Advances in Neural Information Processing Systems*, 36.
- Jiang, J., Dai, Y., Yang, F., & Ma, Z. (2024). A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services. *European Journal of Operational Research*, 312(1), 125–137.
- Jiang, Y., Wu, Y., Cao, Z., & Zhang, J. (2022). Learning to solve routing problems via distributionally robust optimization. In *Proceedings of the AAAI conference on artificial intelligence: vol. 36*, (9), (pp. 9786–9794).
- Jiang, M., & Zhang, Y. (2022). A branch-and-price algorithm for large-scale multidepot electric bus scheduling. *IEEE Transactions on Intelligent Transportation Systems*, 24(12), 15355–15368.
- Jiao, Z., Zhang, H., Zhang, R., & Li, X. (2024). Graph manifold learning with non-gradient decision layer. *Neurocomputing*, Article 127390.
- Kim, M., Park, J., et al. (2021). Learning collaborative policies to solve NP-hard routing problems. *Advances in Neural Information Processing Systems*, 34, 10418–10430.
- Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! In *International conference on learning representations*.
- Kwon, Y. D., Choo, J., Kim, B., Yoon, I., Gwon, Y., & Min, S. (2020). Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 21188–21198.
- Lauri, M., Hsu, D., & Pajarinen, J. (2022). Partially observable markov decision processes in robotics: A survey. *IEEE Transactions on Robotics*, 39(1), 21–40.
- Lei, K., Guo, P., Wang, Y., Wu, X., & Zhao, W. (2022). Solve routing problems with a residual edge-graph attention neural network. *Neurocomputing*, 508, 79–98.
- Lera-Romero, G., Bront, J. J. M., & Soullignac, F. J. (2024). A branch-cut-and-price algorithm for the time-dependent electric vehicle routing problem with time windows. *European Journal of Operational Research*, 312(3), 978–995.
- Li, J., Ma, Y., Gao, R., Cao, Z., Lim, A., Song, W., et al. (2021). Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem. *IEEE Transactions on Cybernetics*, 52(12), 13572–13585.
- Li, C., Zhu, Y., & Lee, K. Y. (2023). Route optimization of electric vehicles based on re-insertion genetic algorithm. *IEEE Transactions on Transportation Electrification*, 9(3), 3753–3768.
- Lu, J., Li, L., & Wang, F. Y. (2024). Event-triggered parallel control using deep reinforcement learning with application to comfortable autonomous driving. *IEEE Transactions on Intelligent Vehicles*, 15(3), 1139–1149.
- Lu, H., Zhang, X., & Yang, S. (2019). A learning-based iterative method for solving vehicle routing problems. In *International conference on learning representations*.
- Luppi, A. I., Mediano, P. A., Rosas, F. E., Holland, N., Fryer, T. D., O'Brien, J. T., et al. (2022). A synergistic core for human brain evolution and cognition. *Nature Neuroscience*, 25(6), 771–782.
- Ma, Y., Cao, Z., & Chee, Y. M. (2023). Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. *Advances in Neural Information Processing Systems*.
- Ma, Y., Li, J., Cao, Z., Song, W., Guo, H., Gong, Y., et al. (2022). Efficient neural neighborhood search for pickup and delivery problems. In *Proceedings of the thirty-first international joint conference on artificial intelligence* (pp. 4776–4784).
- Ma, Y., Li, J., Cao, Z., Song, W., Zhang, L., Chen, Z., et al. (2021). Learning to iteratively solve routing problems with dual-aspect collaborative transformer. *Advances in Neural Information Processing Systems*, 34.
- Ma, R., Wang, Y., Wang, S., Cheng, L., Wang, R., & Tan, M. (2023). Sample-observed soft actor-critic learning for path following of a biomimetic underwater vehicle. *IEEE Transactions on Automation Science and Engineering*, 1–10.
- Optimization, G., & LLC (2022). Gurobi optimizer reference manual. URL <https://www.gurobi.com/>.
- Perron, L., & Furnon, V. (2019). Google OR tools (version 7.2). URL <https://developers.google.com/optimization/>.
- Qian, T., Liu, X. F., & Fang, Y. (2024). A cooperative ant colony system for multi-objective multirobot task allocation with precedence constraints. *IEEE Transactions on Evolutionary Computation*, 1–1.
- Qiu, D., Wang, Y., Ding, Z., & Strbac, G. (2024). Graph reinforcement learning for carbon-aware electric vehicles in power-transport networks. *IEEE Transactions on Smart Grid*.
- Ren, L., Fan, X., Cui, J., Shen, Z., Lv, Y., & Xiong, G. (2022). A multi-agent reinforcement learning method with route recorders for vehicle routing in supply chain management. *IEEE Transactions on Intelligent Transportation Systems*, 23(9), 16410–16420.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, Article 127063.
- Sun, Z., Wang, N., Lin, H., & Zhou, X. (2023). Persistent coverage of UAVs based on deep reinforcement learning with wonderful life utility. *Neurocomputing*, 521, 137–145.
- Sun, Z., & Yang, Y. (2024). Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems*, 36.
- Tamke, F., & Buscher, U. (2021). A branch-and-cut algorithm for the vehicle routing problem with drones. *Transportation Research Part B: Methodological*, 144, 174–203.
- Tang, M., Zhuang, W., Li, B., Liu, H., Song, Z., & Yin, G. (2023). Energy-optimal routing for electric vehicles using deep reinforcement learning with transformer. *Applied Energy*, 350, Article 121711.
- Tian, Q., Zhao, T., & Yuan, R. (2021). An overview of the inequality in China's carbon intensity 1997–2016: a theil index decomposition analysis. *Clean Technologies and Environmental Policy*, 23, 1581–1601.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. *Advances in Neural Information Processing Systems*, 28.
- Wang, C., Ma, B., & Sun, J. (2023). A co-evolutionary genetic algorithm with knowledge transfer for multi-objective capacitated vehicle routing problems. *Applied Soft Computing*, 148, Article 110913.
- Wang, Z., Nie, F., Zhang, C., Wang, R., & Li, X. (2023). Worst-case discriminative feature learning via max-min ratio analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(1), 641–658.
- Wu, Y., Song, W., Cao, Z., Zhang, J., & Lim, A. (2022). Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, 33(9), 5057–5069.
- Xu, Y., Fang, M., Chen, L., Xu, G., Du, Y., & Zhang, C. (2021). Reinforcement learning with multiple relational attention for solving vehicle routing problems. *IEEE Transactions on Cybernetics*, 52(10), 11107–11120.
- Ye, H., Wang, J., Liang, H., Cao, Z., Li, Y., & Li, F. (2024). GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time. In *Proceedings of the AAAI conference on artificial intelligence*.
- Zhang, Z., & Han, Y. (2022). Discrete sparrow search algorithm for symmetric traveling salesman problem. *Applied Soft Computing*, 118, Article 108469.
- Zhang, Z., Liu, H., Zhou, M., & Wang, J. (2023). Solving dynamic traveling salesman problems with deep reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 34(4), 2119–2132.
- Zhang, L., Zhao, J., Lamon, E., Wang, Y., & Hong, X. (2023). Energy efficient multi-robot task allocation constrained by time window and precedence. *IEEE Transactions on Automation Science and Engineering*, 1–12.
- Zheng, J., He, K., Zhou, J., Jin, Y., & Li, C. M. (2023). Reinforced Lin-Kernighan-Helsgaun algorithms for the traveling salesman problems. *Knowledge-Based Systems*, 260, Article 110144.
- Zhou, J., Wu, Y., Cao, Z., Song, W., Zhang, J., & Chen, Z. (2023). Learning large neighborhood search for vehicle routing in airport ground handling. *IEEE Transactions on Knowledge and Data Engineering*, 35(9), 9769–9782.
- Zhou, Y., Xu, W., Fu, Z. H., & Zhou, M. (2022). Multi-neighborhood simulated annealing-based iterated local search for colored traveling salesman problems. *IEEE Transactions on Intelligent Transportation Systems*, 23(9), 16072–16082.
- Zhu, Q., Wu, X., Lin, Q., Ma, L., Li, J., Ming, Z., et al. (2023). A survey on evolutionary reinforcement learning algorithms. *Neurocomputing*, 556, Article 126628.